EXPERIMENT NO. 7

AIM: Create a react application and make use of atleast 4 state hooks available in react. (Simple counter application in react which uses State Hooks)

THEORY:

Introduction

This experiment demonstrates the development of a modular web application using **React.js**, a popular JavaScript library for building user interfaces. The application is structured using reusable components such as Header, Footer, Button, Counter, and TodoItem.
React enables developers to build dynamic and interactive UIs efficiently by dividing the interface into independent, reusable pieces called components. This improves code maintainability, scalability, and readability.

React Components

React applications are built using components, which are independent and reusable blocks of code. Each component represents a part of the user interface.

In this experiment, the following components are used:

- **Header Component** – Displays the top section of the page
- **Footer Component** – Displays the bottom section
- **Counter Component** – Handles dynamic count functionality
- **Button Component** – Reusable buttons for actions like increment/decrement
- **TodoItem Component** – Represents individual tasks

Components can be classified as:

- **Functional Components** – Simple JavaScript functions (used in this experiment)
- **Class Components** – More complex components with lifecycle methods

State Management (useState Hook)

State is used to store and manage dynamic data in a React component. In this experiment, the **useState hook** is used to manage values like count and text.

Example:

- Count value changes when Increment/Decrement buttons are clicked
- Conditional messages like "Number is odd" are displayed based on state

State updates automatically re-render the UI, ensuring real-time updates without refreshing the page.

Event Handling in React

React allows handling user interactions through events such as button clicks.

In this experiment:

- **Increment Button** increases the count
- **Decrement Button** decreases the count
- **Change Button** modifies displayed values
- Checkbox inputs handle user selections

Event handling is done using functions passed as props to components.

Conditional Rendering

Conditional rendering allows displaying different UI elements based on conditions.

Examples from the experiment:

- Displaying "Number is odd" based on count value
- Showing different text like "Value is React" dynamically

This improves user interaction by adapting the UI based on logic.

<u>CODE:</u>

App.js

```
import logo from './logo.svg';
import './App.css';
import Button from './components/Button/Button';
import CounterComponent from './components/CouterComponent/CounterComponent';
import Todoitem from './components/Todoitem/Todoitem';

function App() {
  return (
    <div className="App">
      <h1>Hello World</h1>
      <CounterComponent/>
      <Todoitem text="Eat"/>
      <Todoitem completed={true} text="Code"/>
      <Todoitem text="watch"/>
    </div>
  );
}
```

```
export default App;
```

button.jsx

```
import React from 'react'

function Button() {
  return (
    <div>
      <button className='todo-button' >Click me</button>
    </div>
  )
}

export default Button
```

countercomponent.jsx

```
import React from 'react'
import { useState } from 'react';

function CounterComponent() {
    const[count,setcount]=useState(15);
    const[value,setvalue]=useState("React");
  return (
    <>
      <p>Count Components - {count}</p>
      <h6>Number is {count%2===0?"even":"odd"}</h6>
      <h6>Value is {value}</h6>
      <button onClick={()=>{
        setvalue("meow");
      }}>Change</button>
      <button onClick={()=>setcount(count+1)}>Increment</button>
      <button onClick={()=>setcount(count-1)}>Decrement</button>
    </>
  )
}

export default CounterComponent
```

footer.jsx

```
import React from 'react'

function Footer() {
  return (
    <div>
```

```
    <p>This is the footer</p>
  </div>
 )
}

export default Footer
```

header.jsx

```
import React from 'react'

function Header() {
 return (
  <div>
    <p>This is the header</p>
  </div>
 )
}

export default Header
```

todoitem.jsx

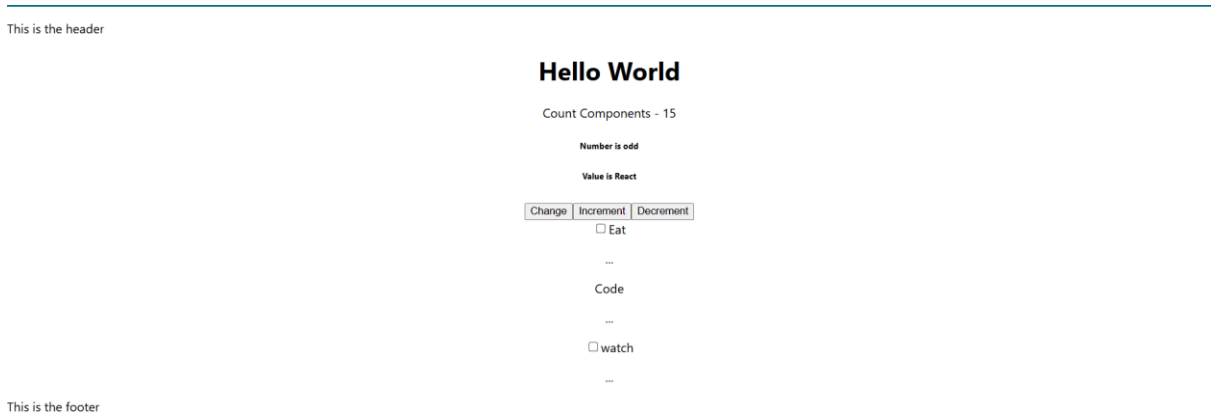
```
import React from 'react'

function Todoitem(props) {
 return (
  <>
    <li className='todo-item'>
      <span>
         {props.completed?<></>:<input type='checkbox' />}
         <span className='todo-itemtext' >{props.text}</span>
      </span>
      <p>...</p>
    </li>
  </>
 )
}

export default Todoitem
```

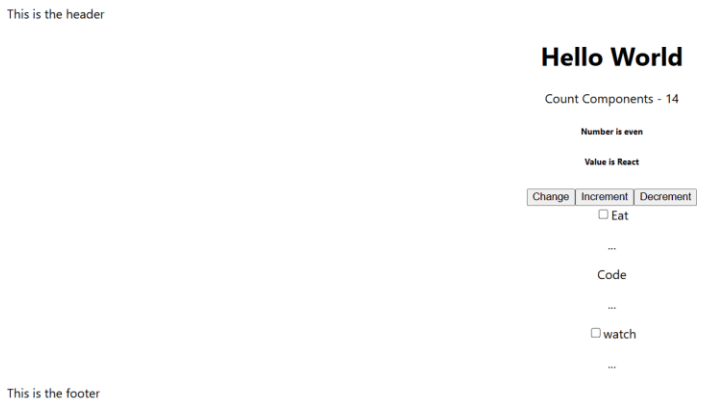
index.js

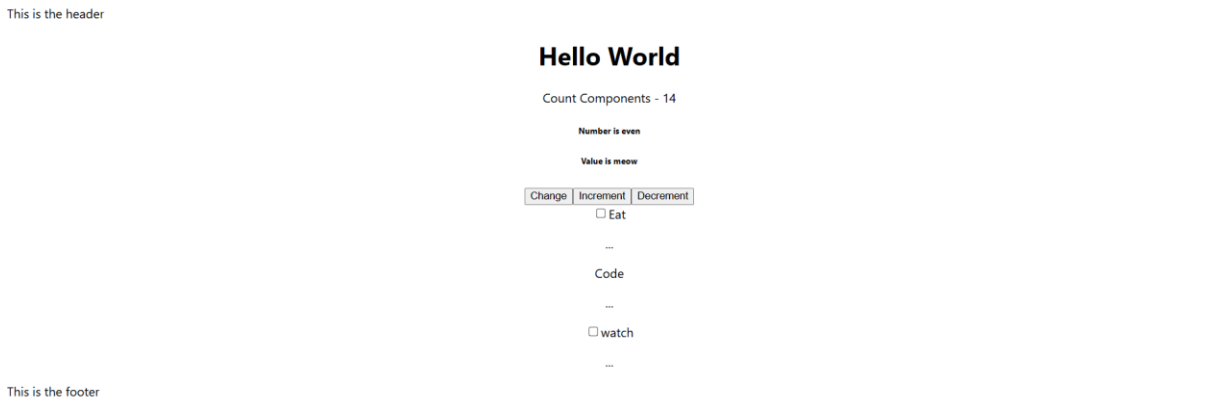
```
import React from 'react';
import ReactDOM from 'react-dom/client';
import './index.css';
import App from './App';
```

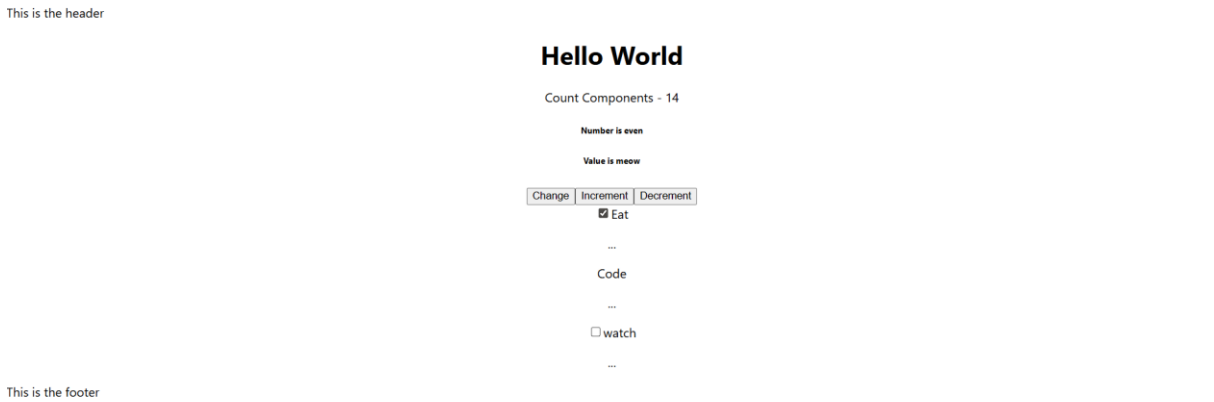
```
import reportWebVitals from './reportWebVitals';
import Header from './components/Header/Header';
import Footer from './components/Footer/Footer';

const root = ReactDOM.createRoot(document.getElementById('root'));

root.render(
  <React.StrictMode>
    <Header/>
    <App />
    <Footer/>
  </React.StrictMode>
);

// If you want to start measuring performance in your app, pass a function
// to log results (for example: reportWebVitals(console.log))
// or send to an analytics endpoint. Learn more: https://bit.ly/CRA-vitals
reportWebVitals();
```

OUTPUT:









<u>CONCLUSION</u>: This experiment highlights the core concepts of React such as component-based architecture, state management, event handling, and conditional rendering. By dividing the UI into reusable components, the application becomes more organized, scalable, and efficient. React's ability to update only necessary parts of the UI enhances performance and user experience.